



07

Kimlik Doğrulama

Kimlik Doğrulama

Kimlik Yöntemleri

MFA

Captcha

Kimlik Doğrulama Sistemi Saldırıları

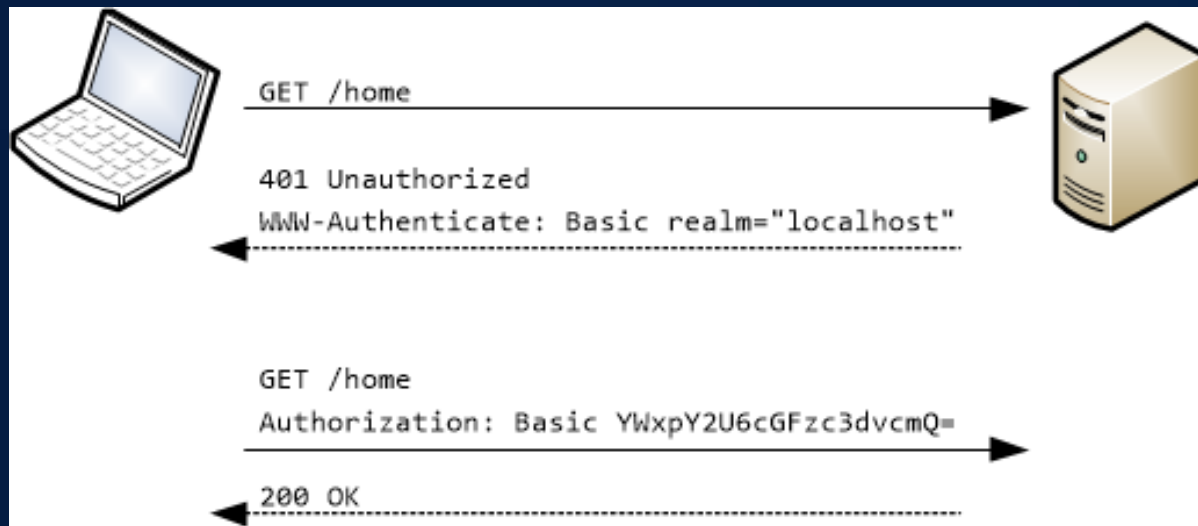
Kimlik Doğrulama



Tarafların gerçekten tanımlı taraflar olup olmadıklarını kontrol etmek için uygulamalarda kimlik doğrulama yapılır.

- Kullanıcının, uygulamaya kendini tanıtmayı
- Uygulamanın, uygulamaya kendini tanıtmayı

Basic Authentication



YWxpY2U6cGFzc3dvcmQ=



alice:password

Basic Authentication

Riskler



Basic Authentication yönteminde kullanılan **Base64** kodlama, şifreleme değildir.



Her istekte Authorization header üzerinden username - password bilgisi gönderildiği için oturum yönetimi söz konusu değildir.



Oturum sonlandırma eksikliği vardır.



Mutlaka **TLS** ile kullanılmalıdır.



Esnek değildir, yetkilendirme yapmak zordur. Gönderilen her istek için kaba kuvvet saldırısı önlemi alınmalıdır.

Token Authentication



Username – password bilgisi karşılığında token alınır.



Kimliklendirme alınan token ile sağlanır.



Özellikle API çağrıları için ideal yapı sunar.

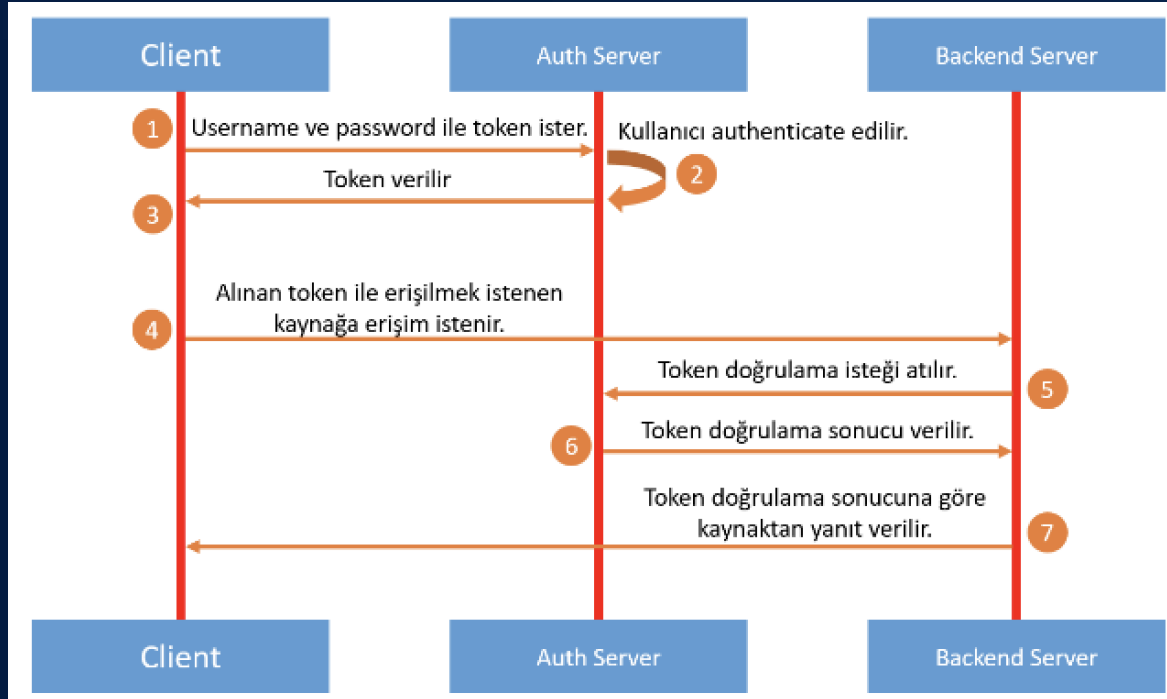


Çok daha esnektir.



Oturum sonlandırma ve oturum yönetimi vardır.

Token Authentication



Token Authentication

JWT



JWT (JSON Web Token), JSON formatında veri içerebilen token yapılarıdır.



3 ana bölümden oluşur.



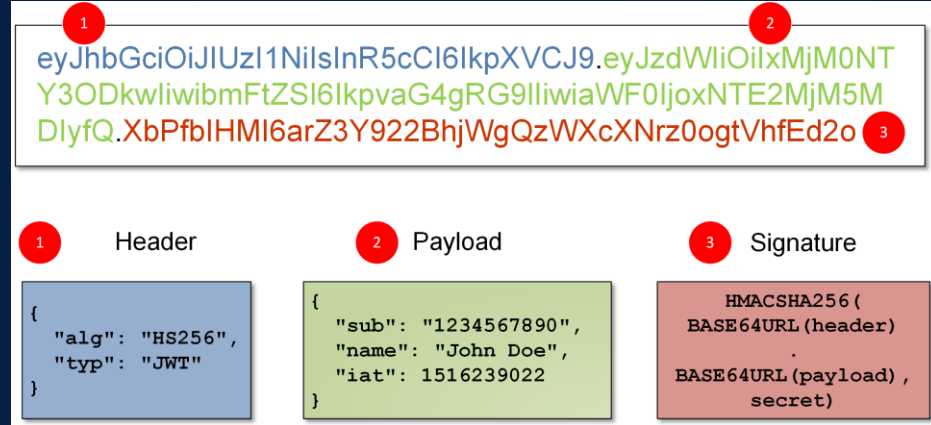
Veri taşıması kolaylık sağlaması ile birlikte bazı risklere neden olabilir.



İçerisinde hassas veri taşınmamalıdır.



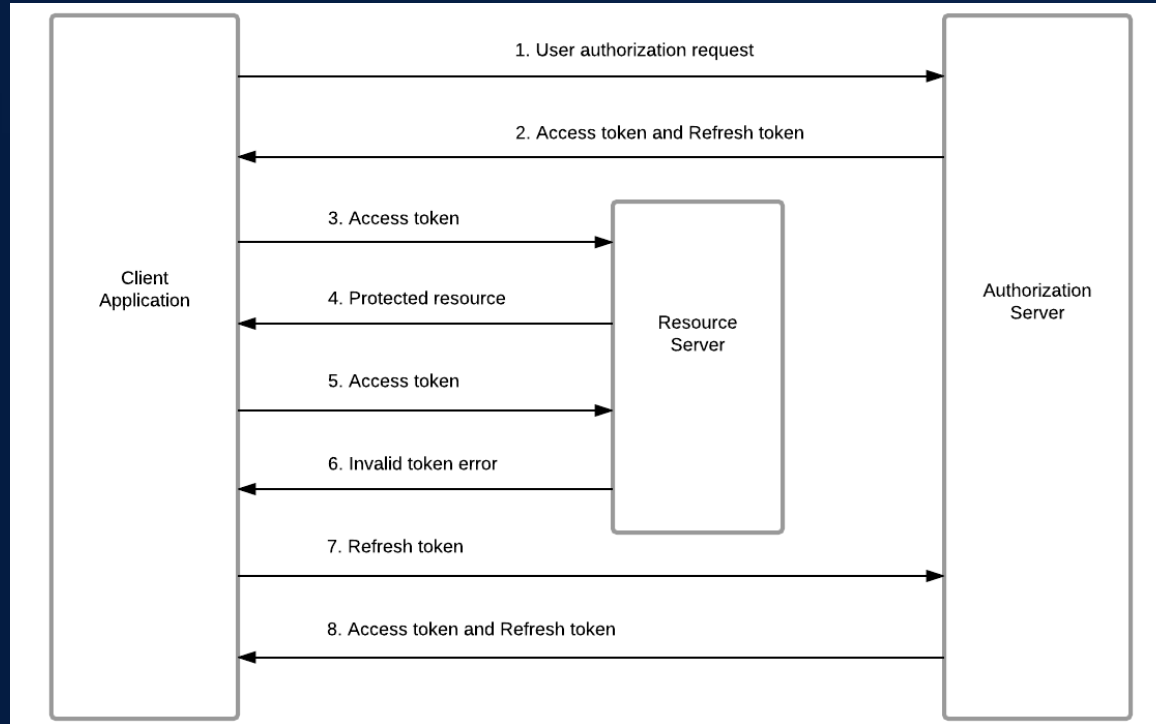
Tokenlar sadece süresi dolduğunda geçersiz olur. Kullanıcı kendi tokenını iptal edemediği için oturum çalınması durumunda risk oluşabilir.



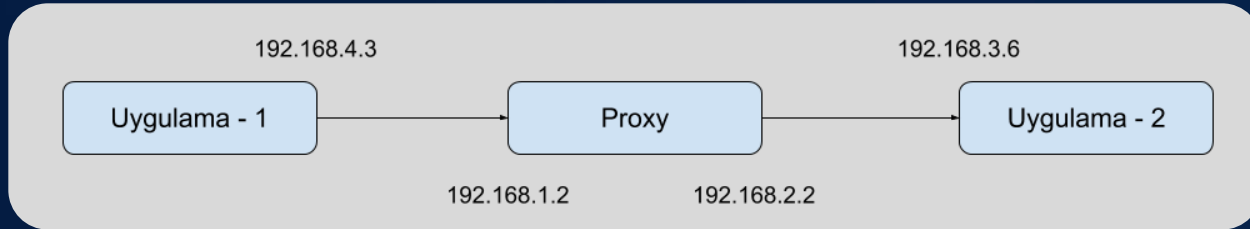
<https://research.securitum.com/jwt-json-web-token-security/>

Token Authentication

Access Token ve Refresh Token



IP Kısıtlama



```
getHeader["x-forwarded-for"];
```

```
192.168.4.3, 192.168.1.2, 192.168.2.2
```

```
getIP();
```

```
192.168.2.2
```

```
GET / HTTP/1.1
User-Agent:
Accept: */*
Postman-Token:
Host:
Accept-Encoding: gzip, deflate
Connection: close
X-Originating-IP: 127.0.0.1
X-Forwarded-For: 127.0.0.1
X-Remote-IP: 127.0.0.1
X-Remote-Addr: 127.0.0.1
```

MFA

(Multi-factor Authentication)



Bir kullanıcının bir sistemde kimlik doğrulaması yapmak için birden fazla kanıt türü sunması gereken yapıdır.



Kritik işlemler içeren uygulamalarda kullanılır.



Bu kanıt türlerini aşağıdaki gibi belirtebiliriz

- Bildiğiniz bir şey (Parola, PIN, güvenlik soruları)
- Sahip olduğunuz bir şey (Donanım veya yazılım tokenları, sertifika, email, SMS)
- Sizden bir şey (Parmak izi, yüz tanıma, göz taraması)
- Lokasyon (Kaynak IP bilgileri, coğrafi konum bilgileri)



<https://www.dignited.com/30668/configure-two-factor-authentication-before-you-get-locked-out-of-your-own-account/>

Captcha



Turing Testi



Önce Captcha, sonra kullanıcı adı – şifre kontrolü



Makine tarafından çözülemeyecek kadar karmaşık



Genellikle kullanıcı oluşturma, şifre resetleme, kullanıcı silme, yorum yazma vb. alanlarda kullanılır.



Kimlik Doğrulama Sistemi Saldırıları



Injection kontrolü eksikliği



Kaba kuvvet (Brute Force)



Varsayılan / basit kullanıcı adı şifre kullanımı



TLS kullanılmaması



Kimlik hırsızlığı
Bilgi ifşası
Yetkisiz Erişim



Uygulama

Brute Force Saldırıları



Kimlik Doğrulama Sistemi Saldırıları

Captcha Zafiyetleri



Görüntü işleme ile çözilememeli



Captcha için olan görüntü seti dar olmamalı



Captcha tek seferlik kontrolde geçerli olmalıdır **Captcha Replay saldırıları**



Farklı fontlar ve karakter bozma yöntemleri içermelidir.

Kimlik Doğrulama Sistemi Saldırıları

Captcha Replay

```
j_username=[REDACTED]&j_password=[REDACTED]&captcha=YT6BQ3&captchaHash=B3A3E912FF94F098A1EDAE1371DCEB49&checkAuth=submit
```



Girilen captcha değerinin session'dan silinmemesidir. Doğru ve yanlış **her denemeden sonra** captcha ve hash değeri **silinmeli**, geçerliliği iptal edilmelidir.

```
String captcha = request.getParameter("captcha");  
c = (Captcha) session.getAttribute(Captcha.NAME);  
//captcha değeri sessiondan silinir.  
session.setAttribute(Captcha.Name, null);  
if (c != null && c.isCorrect(captcha)){  
    // doğru resim  
}  
else {  
    // yanlış resim  
}
```

08

Oturum Yönetimi

Cookie ve Session
CSRF Zafiyeti
Dikkat Edilmesi Gereken Maddeler

Cookie ve Session



HTTP stateless bir protokoldür



Cookie bir kimliktir, session ise bu kimliğe ait bilgilerin tutulduğu objedir

```
POST /login HTTP/1.0
```

```
Host: example.com
```

```
...
```

```
username=test&password=Passw0rd!
```

```
HTTP/1.1 200 OK
```

```
Content-Type: text/html
```

```
Set-Cookie: SessionId=asqDd13cCqDSad123wfasf145lk15jHlk512; path=/;
```

```
GET /profile HTTP/1.0
```

```
Host: example.com
```

```
Cookie: SessionId=asqDd13cCqDSad123wfasf145lk15jHlk512
```

Cookie Özellikleri



Domain, Path



HTTPOnly, Secure



Expires, Max-age

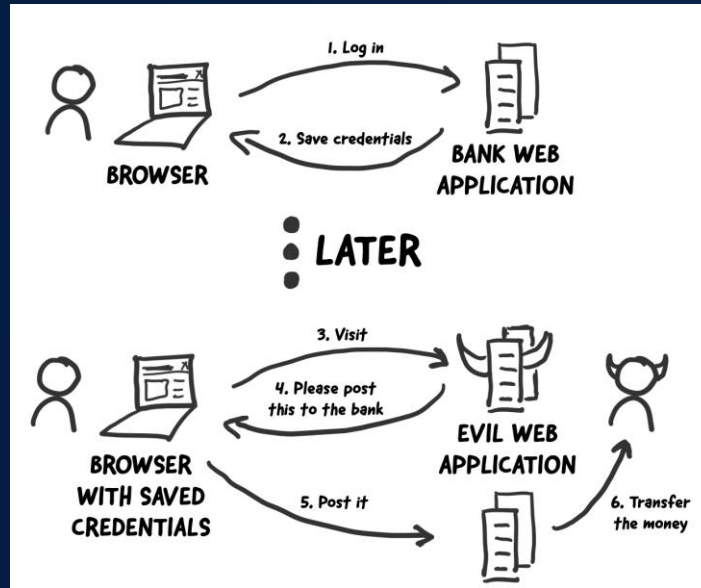


SameSite

CSRF

(Cross-site Request Forgery)

CSRF (Cross-site Request Forgery), A uygulaması üzerinden, B uygulaması üzerinde oturumu açık olan kullanıcının, bilgisi dahilinde olmadan işlem yaptırılabilmesine olanak tanıyan bir güvenlik açığıdır.



CSRF

(Cross-site Request Forgery)

example.com üzerinde mail değiştirme isteği

```
POST /email/change HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 30
Cookie: session=yvthwsztyeQkAPzeQ5gHgTvlyxHfsAfE

email=haktan@example.com
```

```
<html>
<body>
  <form action="https://example.com/email/change"
method="POST">
    <input type="hidden" name="email"
value="hacker@evil-user.net" />
  </form>
  <script>
    document.forms[0].submit();
  </script>
</body>
</html>
```

hacker.com sayfasının kaynak kodu



Uygulama

CSRF

Örnek-1



CSRF

(Cross-site Request Forgery)



CSRF token kullanılarak, sunucu tarafında kontrol edilmelidir.



```
<input type="hidden" name="csrf-token" value="CIwNZNlR4XbisJF39I8yWnWX9wX4WFoz" />
```



CSRF token, oturum başına benzersiz ve tahmin edilemez olmalıdır.



GET metodu üzerinden durum değişikliğine sebep olacak bir işlem yapılmamalıdır.



SameSite cookie: None, Lax, Strict



STRICT



LAX



NONE

Oturum Yönetimi

Dikkat Edilmesi Gereken Maddeler



Kullanılan framework'ün sunduğu oturum denetimleri kullanılmalıdır. Tekerleği yeniden icat etmeye gerek yok 😊



Oturum kimliği oluşturma, güvenilir bir sunucuda yapılmalıdır.



Oturum kimliği, benzersiz, yeterince rastgele üretilmelidir.



Kullanıcıya ait bilgiler, oturum kimliğinde kullanılmamalıdır.



Cookie'lerin domain ve path bilgileri ayarlanmalıdır.

Oturum Yönetimi

Dikkat Edilmesi Gereken Maddeler



Cookie'lerde "Secure" ve "HTTPOnly" bayrakları işaretlenmelidir.



Her başarılı kimlik doğrulama sonrasında, yeni bir oturum oluşturulmalıdır.



Oturumu açmadan önce bir oturum oluşturulduysa, başarılı bir şekilde giriş yapıldıktan sonra yeni bir oturum oluşturulmalıdır.



İş gereksinimine bağlı olarak, mümkün olduğunca kısa bir oturum hareketsizlik süresi belirlenmelidir.



Oturum etkinken bile kalıcı oturum açma işlemlerine izin verilmemeli, periyodik olarak oturum sonlandırması uygulanmalıdır.

Oturum Yönetimi

Dikkat Edilmesi Gereken Maddeler



Mümkünse, aynı kullanıcıya ait eş zamanlı oturum açılabilmesine izin verilmemelidir.



Uygulamadan çıkış yapıldığında, sunucu tarafında ilişkili oturum sonlandırılmalıdır.



Oturum tanımlayıcıları, URL'lerde, hata mesajlarında ve günlüklerde gösterilmemelidir.



Herhangi bir ayrıcalık seviyesi değişikliğinden sonra oturum kimliğini yenilenmelidir.
(Parola değişiklikleri, izin değişiklikleri gibi)



Eğer, durum bilgisinin kullanıcı tarafında saklaması
gerekiyorsa, şifreleme ve bütünlük denetimleri, sunucu tarafında kontrol edilmelidir.

09

Yetkilendirme & Eriřim

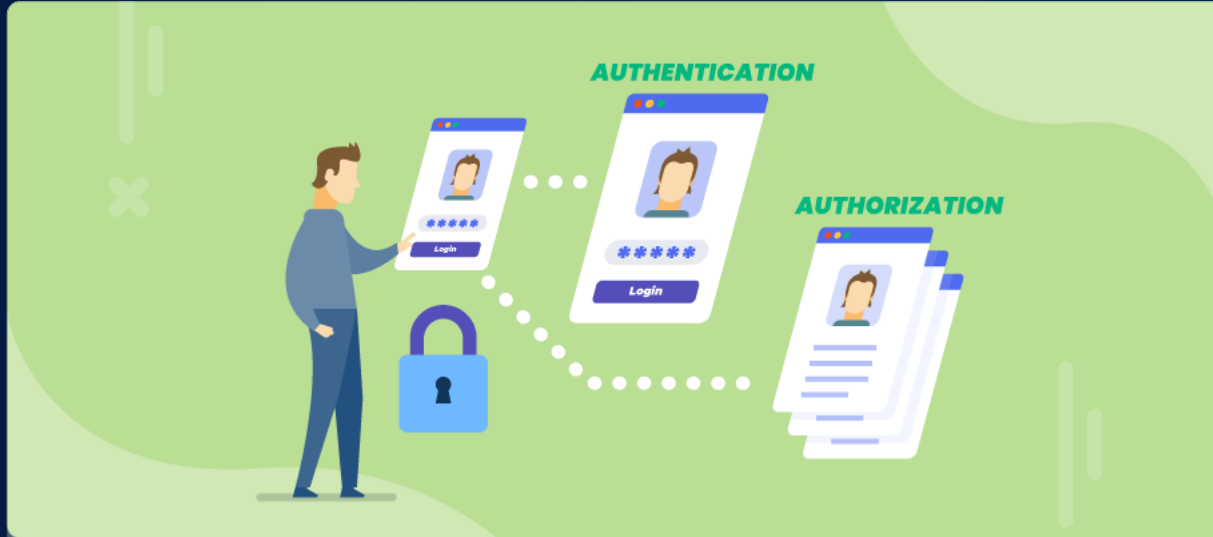
Yetkilendirme

Yatay & Dikey Yetki Yükseltme

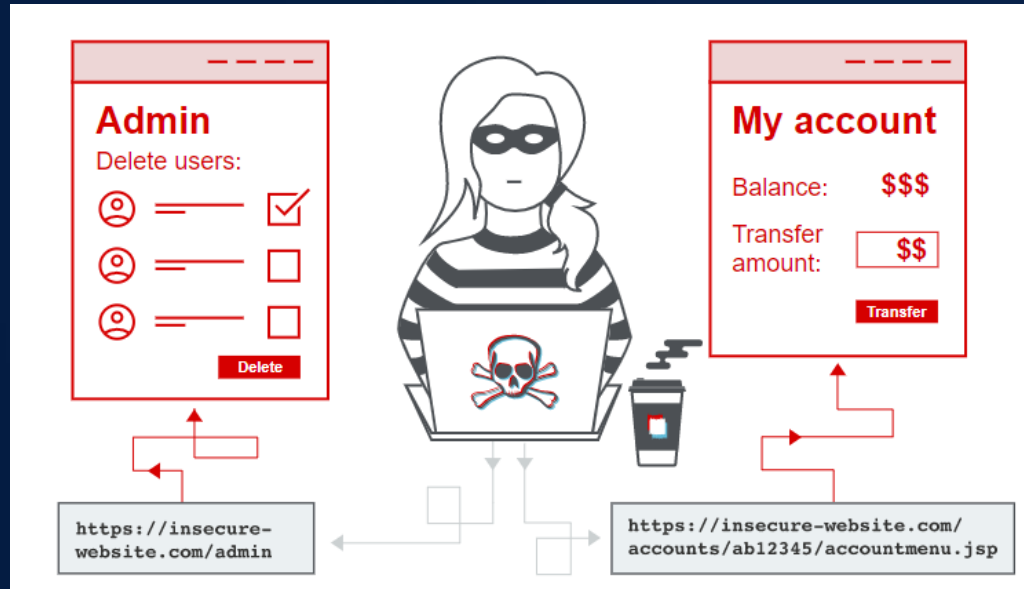
Insecure Direct Object Reference (IDOR) Zafiyeti

Yetkilendirme

Yetkilendirme, kullanıcıların erişim haklarının belirlenmesi ve kontrol edilerek sadece yetkileri dahilindeki kaynaklara erişebilmesidir.



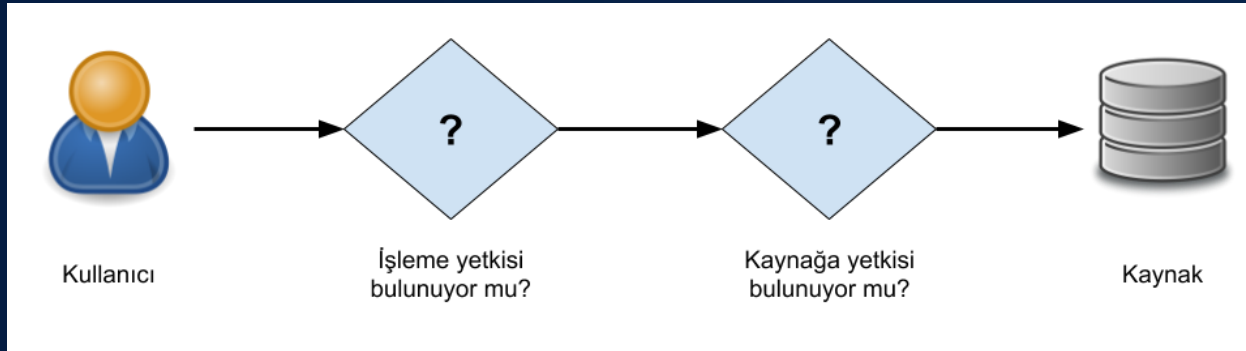
Yatay & Dikey Yetki Yükseltme



IDOR

(Insecure Direct Object Reference)

IDOR (Insecure Direct Object Reference), yetkilendirme kontrolünü eksikliği sebebiyle meydana gelen bir güvenlik açığıdır.



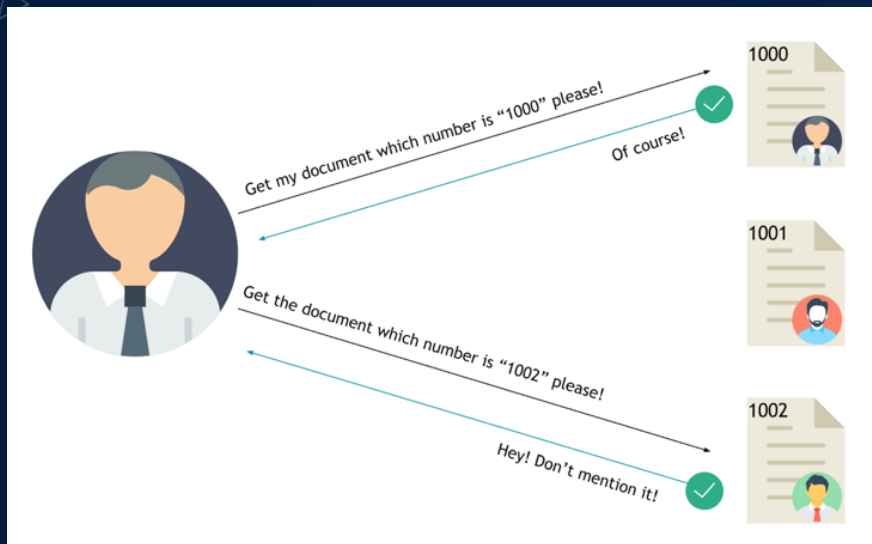
```
pstmt.setString(1, request.getParameter("acct"));
```

```
ResultSet results = pstmt.executeQuery();
```

```
http://example.com/app/accountInfo?acct=notmyacct
```

IDOR

(Insecure Direct Object Reference)



<https://example.com/previewDocument?Id=1000>

<https://example.com/previewDocument?Id=1002>

Kullanıcı	Doküman Id
User1	1000
User2	1002



Broken Function Level Authorization

/api/users/user/myinfo

/api/admins/users/all



Uygulama

IDOR

Örnek-1

Örnek-2

Örnek-3

Örnek-4



IDOR

(Insecure Direct Object Reference)

Önlem



Sunucu tarafında, kullanıcının yapacağı **her işlem** için **yetki kontrolü** sağlanmalıdır. İsteği yapan kullanıcının, erişmek isteğini nesneye erişim yetkisinin olup, olmadığı kontrol edilmelidir.



Uygulama üzerindeki korunaklı kaynaklar (bağlantılar, fonksiyonlar, nesneler vs.) sadece yetkili kullanıcılar tarafından erişilebilecek şekilde kısıtlanmalıdır.



Uygulama mekanizmaları varsayılan olarak tüm erişimleri reddetmeli, sadece uygun olan role yönelik işlemlere izin verilmelidir.

10

İş Mantığı Zafiyetleri



İş Mantığı Zafiyetleri



<https://www.indusface.com/blog/business-logic-vulnerabilities/>

Örnek-1

Örnek-2

Örnek-3

11

API Security

Modern Web App Mimarisi
Top 10 API Güvenlik Açıkları
API Güvenliği - Lab çözümleri



Giriş

API (Application Programming Interface)
REST API, SOAP API, GraphQL API



InfoSec Insider

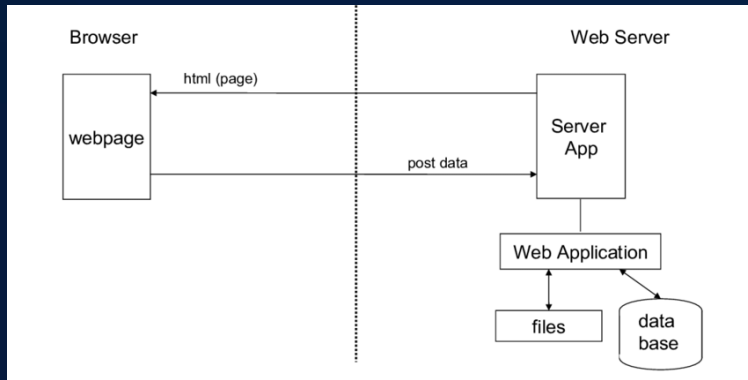
Tony Lauro

January 24, 2019

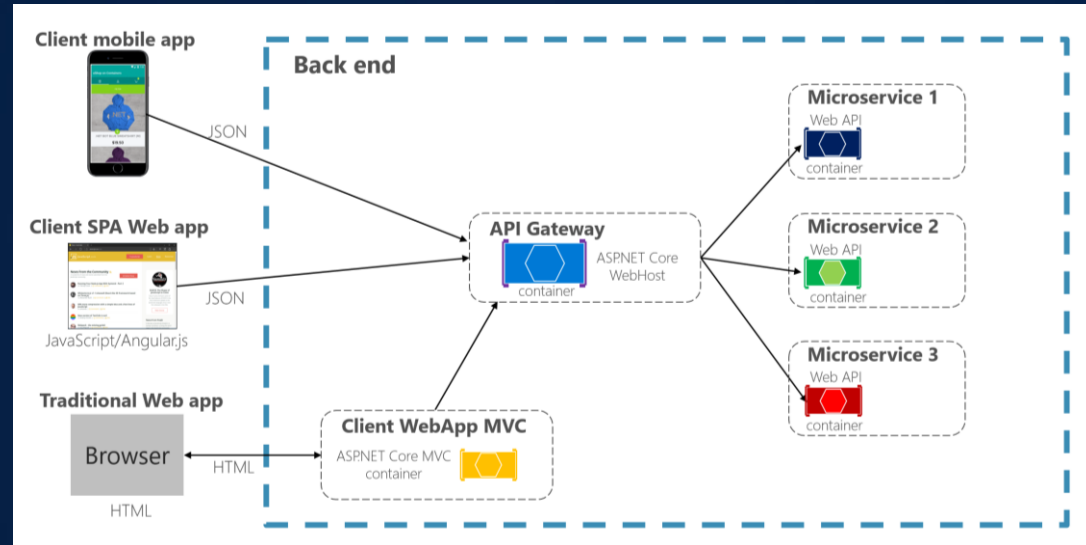
A look at API attack trends such as the current (and failing) architectural designs for addressing security of these API transactions.

Akamai research shows that 83 percent of all traffic on the web today are API calls (JSON / XML). In many cases this fast growth can be attributed to the adoption and popularity of mobile devices and the mobile app ecosystem, as well as the abuse by threat actors using

API Gateway - Modern Web App Mimarisi



Geleneksel Web App Mimarisi



API Gateway ve Micro Servis Mimarisi

API Security Top 10 2023

OWASP API Security Top-10 2019

API1 Broken Object Level Authorization
API2 Broken User Authentication
API3 Excessive Data Exposure
API4 Lack of Resources & Rate Limiting
API5 Broken Function Level Authorization
API6 Mass Assignment
API7 Security Misconfiguration
API8 Injection
API9 Improper Assets Management
API10 Insufficient Logging & Monitoring

OWASP API Security Top-10 2023

API1 Broken Object Level Authorization	Same
API2 Broken Authentication	Updated
API3 Broken Object Property Level Authorization	Updated
API4 Unrestricted Resource Consumption	Updated
API5 Broken Function Level Authorization	Same
API6 Unrestricted Access to Sensitive Business Flows	New
API7 Server-Side Request Forgery (SSRF)	New
API8 Security Misconfiguration	Same
API9 Improper Inventory Management	Updated
API10 Unsafe Consumption of APIs	New

API Test - Araçlar



POSTMAN



SoapUI



Burp Suite

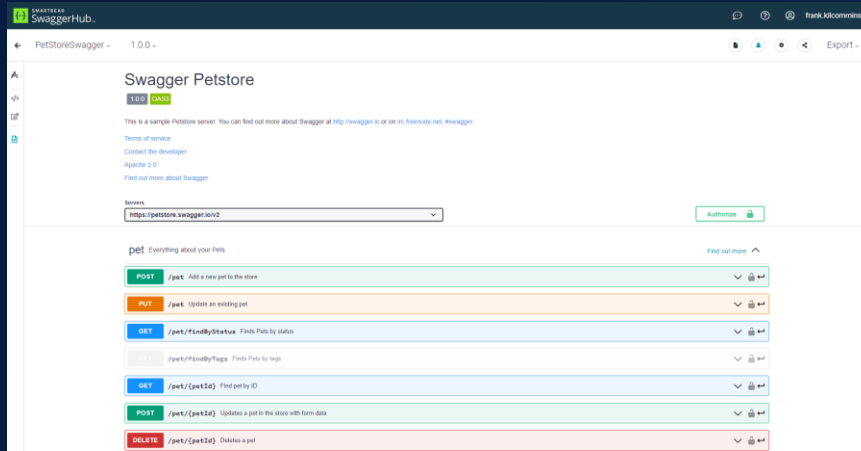
JS Link Finder: <https://github.com/InitRoot/BurpJSLinkFinder>

JS Miner: <https://github.com/minamo7sen/burp-JS-Miner>

Param Miner: <https://portswigger.net/bappstore/17d2949a985c4b7ca092728dba871943>

API Recon

Dokümantasyon sayfaları



<https://petstore.swagger.io/>

API Directory

- [Postman Explore](#)
- [SwaggerHub](#)
- [RapidAPI Hub](#)
- [APIs Guru](#)

Swagger tespiti için wordlist : <https://github.com/danielmiessler/SecLists/blob/master/Discovery/Web-Content/swagger.txt>

API Security Uygulamalar



WEB SECURITY
ACADEMY

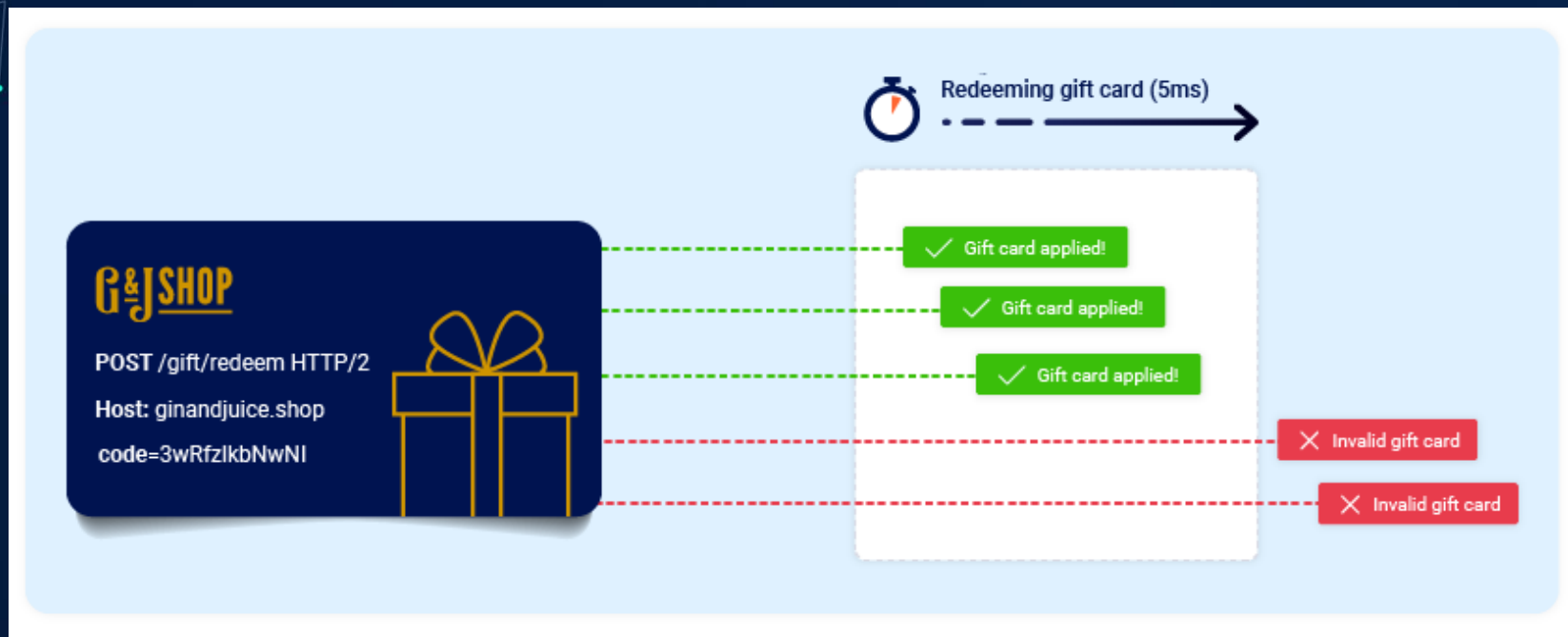
Örnek-1
Örnek-2
Örnek-3
Örnek-4
Örnek-5

12

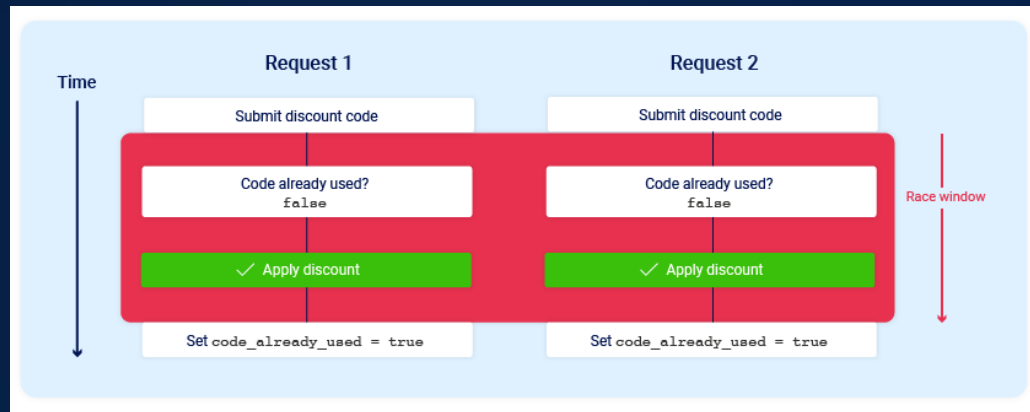
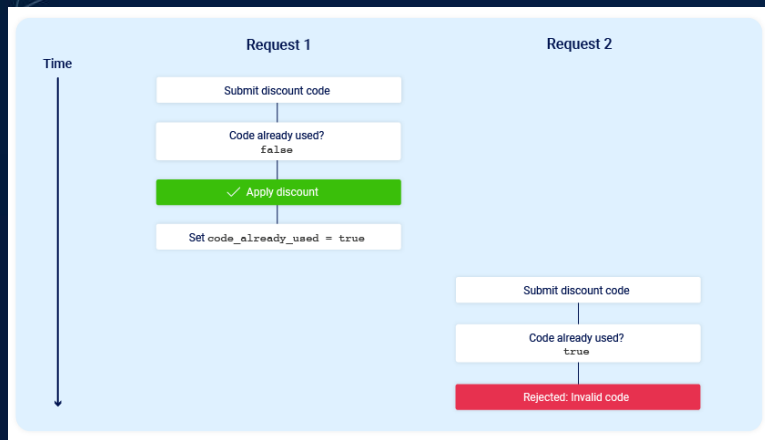
Race Condition



Race Condition



Race Condition



Race Condition Lab



Örnek - 1

Örnek - 2

Korunma Yöntemleri



Thread lock mekanizmaları kullanılmalı



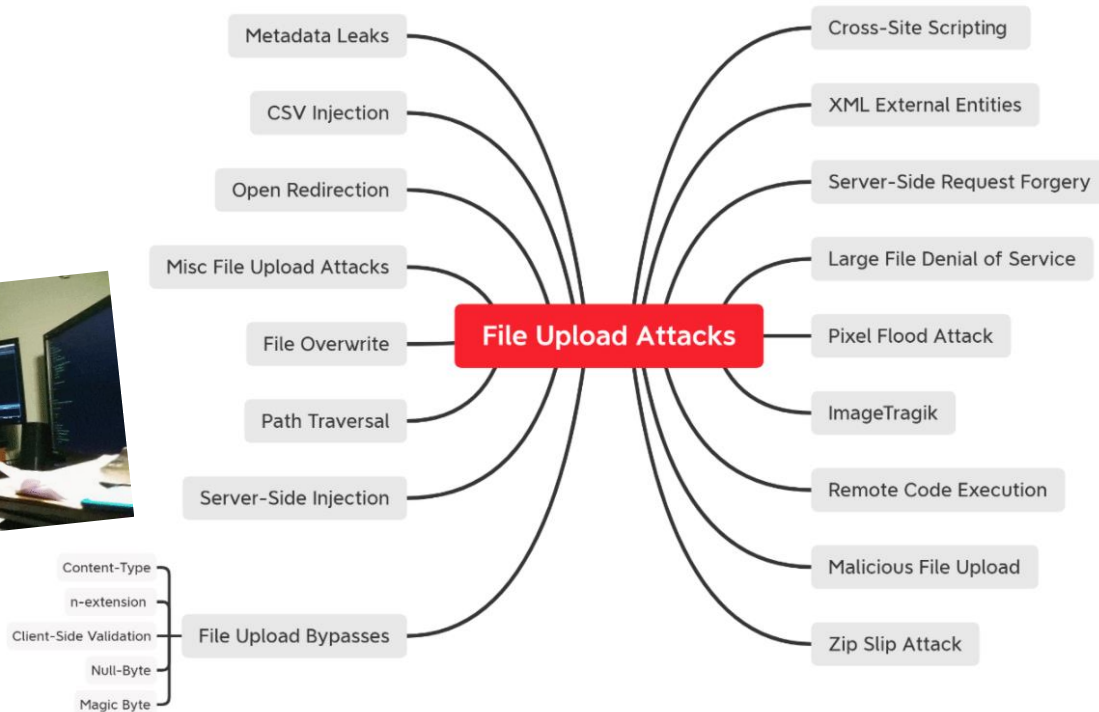
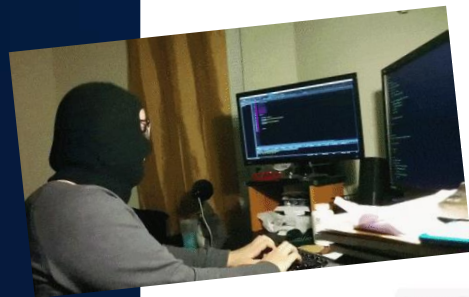
Kritik bölge kontrolleri uygulanmalı

13

Dosya Yükleme Saldırıları



Dosya Yükleme Saldırıları



Güvensiz Dosya Yükleme



Zararlı dosyaların karşıdan yüklenebilmesi ve sunucu üzerinde çalıştırılması ile yapılan saldırılardır.



Dosya Yükleme Saldırıları

Atlatma Teknikleri

Client-side Kontrol



```
file.Php  
file.php7
```



```
file.jpg.php  
file.php.jpg  
file.php%00
```

Blacklist



```
application/x-httpd-php > image/jpeg
```

Content-Type Kontrolü

Magic Byte Kontrolü



Hex:

JPG: FF D8 FF EE

PNG: 89 50 4E 47 0D 0A 1A 0A

GIF: 47 49 46 38 39 61

Dosya Yükleme Saldırıları

Dosya Adı Üzerinden Oluşabilecek Zafiyetler

```
file$(whoami).png # Command Injection
file';select+sleep(10);--.png # SQLi
file"><script>alert(1)</script>.png # Cross-site Scripting (XSS)
```

```
../../../../file.jpg
../../../../etc/passwd
```

Zip Slip

```
../../../../root/.ssh/authorized_keys
dosya.txt
```

Dosya Yükleme Saldırıları

Potansiyel Atak Vektörleri

Denial of Service (DOS)
Metadata'ların Açığa Çıkması
XML External Entity (XXE) Injection
Zafiyetli Bileşen Kullanımı



<https://hackerone.com/reports/446238>



<https://hackerone.com/reports/135072>



<https://hackerone.com/reports/1062888>

Dosya Yükleme Saldırıları

Örnek - SVG dosyası üzerinden XSS

```
<?xml version="1.0" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN"
"http://www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg version="1.1" baseProfile="full"
xmlns="http://www.w3.org/2000/svg">
  <polygon id="triangle" points="0,0 0,50 50,0" fill="#009900"
stroke="#004400"/>
  <script type="text/javascript">
    alert(document.domain);
  </script>
</svg>
```



Dosya Yükleme Saldırıları

Önlemler



İş ihtiyacı kapsamında gerekli olan dosya uzantıları belirlenerek, beyaz liste yaklaşımı ile kontrol edilmelidir. Sadece istenilen dosya uzantılarına izin verilmelidir.



Yüklenen dosyanın içeriği kontrol edilmelidir.



Yüklenebilecek dosyaların boyutu sınırlandırılmalıdır.



Uygulama tarafında yüklenen dosyaların adları değiştirilmelidir.



Sadece yetkili kişiler tarafından dosya yüklenebilmesine izin verilmelidir.



Yüklenen dosyalara erişim sağlanırken yetki kontrollerinin uygun bir şekilde yapıldığından emin olunmalıdır.

Dosya Yükleme Saldırıları

Önlemler



Dosya izinleri en az ayrıcalıklar ilkesine göre ayarlanmalıdır.



Yüklenen dosyaların, zararlı bir içeriğe sahip olup olmadığını doğrulamak için bir anti virüs veya sandbox çözümü kullanılabilir.



Kullanılan kütüphaneler güncel tutulmalı ve güvenli bir şekilde yapılandırılmalıdır.



Uygulama

Güvensiz Dosya Yükleme

Örnek-1

Örnek-2

Örnek-3

